LLMsVerifier_COMPREHENSIVE_ANALYSIS

LLMsVerifier - Comprehensive Analysis Report

Prepared for: HelixTranslate Integration

A. PROJECT OVERVIEW

Language & Framework

- **Primary Language**: Go (Golang) 1.25.3
- **Secondary Languages**: TypeScript/JavaScript (web UI), Python (SDK), Dart (mobile), Rust (messaging)
- **UI Frameworks**:
  - CLI: Cobra (command-line framework) + Bubble Tea (TUI)
  - Web: Next.js (website/)
  - Mobile: Flutter, React Native, Aurora OS, Harmony OS
  - Desktop: Electron, Tauri

Purpose

LLMsVerifier is an **enterprise-grade LLM verification and benchmarking platform**. Its primary purpose is to:

1. **Verify LLMs** - Confirm models can "see" and understand provided code ("Do you see my code?" test)
2. **Benchmark LLMs** - Measure response time, capability, and performance across providers
3. **Score & Rank LLMs** - Comprehensive scoring system with weighted components
4. **Monitor LLMs** - Real-time health checking with intelligent failover
5. **Generate Verified Configs** - Export configurations containing only verified models

Architecture

- **Monolithic Go application** with modular package structure
- **SQLite database** with SQLCipher encryption (WAL mode, optimized pragmas)
- **REST API server** (Gin framework) with JWT authentication
- **3-tier model discovery**: User config -> Provider API -> models.dev fallback
- **Provider adapter pattern** for 25+ LLM providers
- **Pluggable scoring strategy** system
- **Docker/Kubernetes ready** with health checks and monitoring

---

B. COMPLETE DIRECTORY STRUCTURE

```
LLMsVerifier/
├── go.mod                          # Root Go module (imports llm-
verifier submodule)
├── package.json                    # NPM metadata (minimal)
├── Dockerfile                      # Multi-stage build with
distroless runtime
├── docker-compose.yml              # Docker Compose with health
checks
├── docker-compose.prod.yml         # Production compose
├── docker-compose.messaging.yml    # Messaging stack (Kafka/
RabbitMQ)
├── k8s-manifests.yaml              # Kubernetes manifests
├── Makefile                        # Build automation
├── README.md                       # Main documentation
├── API_REFERENCE.md                # API reference documentation
├── ACP_API_DOCUMENTATION.md        # ACP (AI Coding Protocol) API
docs
├── ACP_*.md                        # Multiple ACP documentation
files
├── VERIFYING.md                    # Verification guide
├── AGENTS.md                       # CLI agents documentation
├── SECURITY.md                     # Security documentation
├── CONSTITUTION.md                 # Project constitution
├── CHALLENGES_*.md                 # Challenge system
documentation
├── VERIFICATION_*.md               # Verification documentation
├── SCORING_*.md                    # Scoring system documentation
├── DEPLOYMENT.md                   # Deployment guide
├── docs/                           # Documentation directory
│   ├── ARCHITECTURE_OVERVIEW.md
│   ├── COMPLETE_SYSTEM_DOCUMENTATION.md
│   ├── MODEL_VERIFICATION_GUIDE.md
│   ├── LLMSVD_SUFFIX_GUIDE.md
│   ├── CONFIGURATION_MIGRATION_GUIDE.md
│   ├── CAPABILITY_DETECTION.md
│   ├── monitoring/
│   ├── protocols/
│   ├── releases/
│   └── scoring/
```

```
│
├── llm-verifier/                    # MAIN APPLICATION CODE
│   ├── go.mod                       # Application Go module
│   ├── go.sum                       # Dependency checksums
│   ├── cmd/main.go                  # Main entry point
│   ├── config.yaml                  # Configuration file
│   ├── config.yaml.example          # Configuration example
│   │
│   ├── api/                         # REST API (Gin-based)
│   │   ├── server.go                # API server setup
│   │   ├── handlers.go              # HTTP handlers (487 lines)
│   │   ├── middleware.go            # Auth & rate limiting
middleware
│   │   ├── validation.go            # Request validation (custom
validators)
│   │   ├── errors.go                # API error types
│   │   ├── sanitize.go              # Input sanitization
│   │   ├── audit_logger.go          # Audit logging
│   │   ├── compliance_checker.go    # Compliance checks
│   │   ├── content_filter.go        # Content filtering
│   │   ├── schema_validator.go      # Schema validation
│   │   └── docs/docs.go             # Swagger/OpenAPI docs
│   │
│   ├── llmverifier/                 # CORE VERIFICATION ENGINE
│   │   ├── verifier.go              # Main Verifier struct
│   │   ├── models.go                # Data models
(VerificationResult, ModelInfo, etc.)
│   │   ├── llm_client.go            # LLM client for testing
│   │   ├── reporter.go              # Report generation (Markdown/
JSON)
│   │   ├── strategy.go              # Scoring strategy interface
│   │   ├── strategy_builder.go      # Strategy builder (fluent
API)
│   │   ├── strategy_default.go      # Default scoring strategy
│   │   ├── config_loader.go         # Configuration loading
(Viper)
│   │   ├── config_export.go         # Config export for CLI agents
│   │   ├── config_migration.go      # Config migration
│   │   ├── issue_detector.go        # Issue detection
│   │   ├── analytics.go             # Analytics
│   │   └── recipes/                 # Test recipes
│   │       ├── recipe.go
│   │       ├── context_window.go
│   │       ├── streaming.go
│   │       ├── vision.go
│   │       ├── instruction.go
│   │       └── long_session.go
│   │
│   ├── providers/                   # PROVIDER ADAPTERS (25+
providers)
│   │   ├── base.go                  # BaseAdapter (common HTTP
functionality)
│   │   ├── service.go               # Provider service interface
│   │   ├── config.go                # ProviderConfig,
```

```
ProviderRegistry
│   │   ├── errors.go              # ProviderError,
ErrorClassifier
│   │   ├── http_client.go         # Provider HTTP client
│   │   ├── model_provider_service.go     # 3-tier model discovery
│   │   ├── model_verification_service.go # Model verification
service
│   │   ├── verified_config_generator.go  # Generate verified
configs
│   │   ├── fallback_models.go     # Fallback model lists
│   │   ├── config_validator.go    # Config validation
│   │   ├── openai.go              # OpenAI adapter
│   │   ├── openai_endpoints.go    # OpenAI API endpoints
│   │   ├── anthropic.go           # Anthropic adapter (with
OAuth support)
│   │   ├── groq.go                # Groq adapter
│   │   ├── deepseek.go            # DeepSeek adapter
│   │   ├── cohere.go              # Cohere adapter
│   │   ├── mistral.go             # Mistral adapter
│   │   ├── xai.go                 # xAI/Grok adapter
│   │   ├── replicate.go           # Replicate adapter
│   │   ├── cerebras.go            # Cerebras adapter
│   │   ├── cloudflare.go          # Cloudflare Workers AI
│   │   ├── siliconflow.go         # SiliconFlow adapter
│   │   ├── hyperbolic.go          # Hyperbolic adapter
│   │   ├── togetherai.go          # Together AI adapter
│   │   ├── sambanova.go           # SambaNova adapter
│   │   ├── kilo.go                # KiloCode adapter
│   │   ├── kimi.go                # Moonshot/Kimi adapter
│   │   ├── qwen.go                # Qwen/DashScope adapter
│   │   ├── novita.go              # Novita AI adapter
│   │   ├── nlpcloud.go            # NLP Cloud adapter
│   │   ├── nia.go                 # NIA adapter
│   │   ├── upstage.go             # Upstage adapter
│   │   ├── sarvam.go              # Sarvam adapter
│   │   ├── publicai.go            # Public AI adapter
│   │   ├── zhipu.go               # Zhipu adapter
│   │   ├── modal.go               # Modal adapter
│   │   ├── vulavula.go            # Vulavula adapter
│   │   ├── kimicode.go            # KimiCode adapter
│   │   └── verification_integration_example.go
│   │
│   ├── verification/              # VERIFICATION SYSTEM
│   │   ├── verification.go        # Core verification logic
│   │   ├── code_verification.go   # Mandatory code visibility
verification
│   │   ├── coding_capability_verification.go  # Coding capability
tests
│   │   ├── provider_client.go     # Provider client interface
│   │   ├── models_dev_enhanced.go  # Enhanced models.dev client
│   │   └── verification_real.go   # Real-world verification
│   │
│   ├── scoring/                   # SCORING SYSTEM
│   │   ├── scoring_engine.go      # ScoringEngine - calculates
```

```
scores
│   │   ├── types.go                # Score types (ScoreWeights,
ComprehensiveScore)
│   │   ├── api_handlers.go         # Scoring API handlers
│   │   ├── metrics_collector.go    # Metrics collection
│   │   ├── alert_manager.go        # Alert management
│   │   ├── model_display.go        # Model display names
│   │   ├── model_naming.go         # Model naming conventions
│   │   ├── monitoring.go           # Scoring monitoring
│   │   ├── database_integration.go # Database integration
│   │   └── main.go                 # Scoring service main
│   │
│   ├── capabilities/               # CAPABILITY DETECTION
│   │   ├── detector.go             # Dynamic capability detector
│   │   ├── types.go                # Capability types & enums
│   │   ├── registry.go             # Provider capability registry
│   │   └── config_generator.go     # Config generation from
capabilities
│   │
│   ├── database/                   # DATABASE LAYER (SQLite +
SQLCipher)
│   │   ├── database.go             # Database struct & encryption
setup
│   │   ├── crud.go                 # CRUD operations
│   │   ├── migrations.go           # Schema migrations
│   │   ├── validation.go           # SQL injection prevention
│   │   ├── optimizations.go        # Query optimizations
│   │   ├── in_memory.go            # In-memory store
│   │   ├── api_keys_crud.go        # API key CRUD
│   │   ├── config_exports_crud.go  # Config export CRUD
│   │   ├── events_crud.go          # Events CRUD
│   │   ├── issues_crud.go          # Issues CRUD
│   │   ├── logs_crud.go            # Logs CRUD
│   │   ├── pricing_crud.go         # Pricing CRUD
│   │   ├── schedules_crud.go       # Schedules CRUD
│   │   ├── users_crud.go           # Users CRUD
│   │   └── verification_scores_crud.go
│   │
│   ├── config/                     # CONFIGURATION SYSTEM
│   │   ├── config.go               # Config struct definitions
│   │   ├── validation.go           # Config validation
│   │   ├── validator.go            # Validator setup
│   │   ├── production_config.go    # Production defaults
│   │   └── examples/               # Example configs
│   │
│   ├── client/                     # API CLIENT
│   │   ├── client.go               # REST API client
│   │   ├── http_client.go          # HTTP client with Brotli
detection
│   │   ├── client_manager.go       # Client manager
│   │   ├── rate_limiter.go         # Rate limiting
│   │   └── analytics.go            # Client analytics
│   │
│   ├── auth/                       # AUTHENTICATION &
```

```
AUTHORIZATION
│   │   ├── auth_manager.go         # JWT + Argon2 + RBAC
│   │   ├── ldap.go                 # LDAP integration
│   │   ├── rbac.go                 # Role-based access control
│   │   ├── oauth_stub.go           # OAuth stub
│   │   └── compliance.go           # Auth compliance
│   │
│   ├── cmd/                        # ADDITIONAL ENTRY POINTS
│   │   ├── acp-cli/                # ACP CLI tool
│   │   ├── code-verification/      # Code verification tool
│   │   ├── crush-config-converter/ # Crush config converter
│   │   ├── full-verify/            # Full verification
│   │   ├── model-verification/     # Model verification
│   │   ├── partners/               # Partner management
│   │   ├── quick-verify/           # Quick verification
│   │   ├── test-direct/            # Direct testing
│   │   ├── test-models-live/       # Live model testing
│   │   ├── testsuite/              # Test suite runner
│   │   ├── tui/                    # Terminal UI
│   │   └── ultimate-challenge/     # Ultimate challenge
│   │
│   ├── api_keys/                   # API KEY MANAGEMENT
│   │   ├── manager.go              # API key manager
│   │   ├── env_scanner.go          # Environment scanner
│   │   └── priority.go             # Key priority system
│   │
│   ├── tui/                        # TERMINAL UI
│   │   └── screens/                # TUI screens
│   │
│   ├── web/                        # WEB APPLICATION
│   │   └── src/app/                # Next.js app
│   │
│   ├── sdk/                        # SDKs
│   │   ├── go/                     # Go SDK
│   │   ├── python/                 # Python SDK
│   │   └── javascript/             # JavaScript SDK
│   │
│   ├── enhanced/                   # ENTERPRISE FEATURES
│   │   ├── adapters/               # Enterprise adapters
│   │   ├── analytics/              # Predictive analytics
│   │   ├── checkpointing/          # State checkpointing
│   │   ├── context/                # Context management (24hr+
sessions)
│   │   ├── enterprise/             # Enterprise features
│   │   ├── supervisor/             # Supervisor/worker pattern
│   │   ├── validation/             # Enhanced validation
│   │   └── vector/                 # Vector database integration
│   │
│   ├── messaging/                  # MESSAGING SYSTEM
│   │   └── factory/                # Message broker factory
│   │
│   ├── monitoring/                 # MONITORING
│   │   └── grafana/                # Grafana dashboards
│   │
```

```
│   ├── performance/                # PERFORMANCE TESTING
│   ├── security/                   # SECURITY FEATURES
│   ├── failover/                   # FAILOVER MECHANISMS
│   ├── scheduler/                  # JOB SCHEDULER
│   ├── notifications/              # NOTIFICATION SYSTEM
│   ├── multimodal/                 # MULTIMODAL SUPPORT
│   ├── events/                     # EVENT SYSTEM
│   └── logging/                    # LOGGING SYSTEM
│
├── sdk/                            # EXTERNAL SDKs
│   ├── python/llm_verifier_sdk/    # Python SDK package
│   └── java/                       # Java SDK
│
├── configs/                        # Configuration templates
├── tests/                          # Test suites
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   ├── performance/
│   ├── security/
│   └── automation/
│
├── helm/                           # Helm charts
├── k8s/                            # Kubernetes configs
├── Website/                        # Website source
├── website/                        # Alternative website
├── mobile/                         # Mobile app variants
└── scripts/                        # Utility scripts
```

C. FULL API SURFACE MAP

C.1 Core Package: `digital.vasic.llmsverifier/llmverifier`

Types

```
// Main types
type Verifier struct { /* config, timing */ }
type VerificationResult struct {
    ModelInfo, Availability, ResponseTime, FeatureDetection,
    CodeCapabilities, GenerativeCapabilities, PerformanceScores,
    Timestamp, Error, ScoreDetails
}
type ModelInfo struct { /* ID, Endpoint, Capabilities,
        ContextWindow, Prices, etc. */ }
type PerformanceScore struct { /* OverallScore, Responsiveness,
        CodeCapability, etc. */ }
type ScoreDetails struct { /* detailed scoring breakdown */ }
type Summary struct { /* TotalModels, AvailableModels,
        FailedModels, AverageScore, etc. */ }
type TopPerformer struct { /* ModelName, Score, Rank */ }
```

```
type ConversationSummary struct { /* Summary, Topics, Sentiment,
        KeyPoints */ }
type LLMClient struct { /* HTTP client for LLM APIs */ }
type WeightConfig struct { /* Responsiveness, CodeCapability,
        FeatureRichness, Reliability, VisionCapability,
        InstructionFollowing */ }
type TestResult struct { /* TestID, Category, Passed, Score,
        Latency, Error, Details */ }
type StrategyScore struct
        { /* Overall, Breakdown, Passed, Details */ }
type Thresholds struct { /* MinOverallScore, MaxLatency,
        MinContextWindow, RequiredCapabilities */ }

// Interfaces
type ScoringStrategy interface {
    Name() string
    Description() string
    WeightConfig() WeightConfig
    CustomTests() []VerificationTest
    ScoreModel(ctx, model, results) (StrategyScore, error)
    FilterModels(models) []ModelInfo
    MinimumThresholds() Thresholds
}
type VerificationTest interface {
    ID() string; Name() string; Category() TestCategory
    Run(ctx, client) (TestResult, error)
    Weight() float64; Required() bool
}
```

Functions/Methods

```
// Verifier
func New(cfg *config.Config) *Verifier
func (v *Verifier) StartTiming() / EndTiming() / GetStartTime() /
        GetEndTime()
func (v *Verifier) GetGlobalClient() *LLMClient
func (v *Verifier) SummarizeConversation(messages []string)
        (*ConversationSummary, error)
func (v *Verifier) Verify() ([]VerificationResult, error)
func (v *Verifier) GenerateMarkdownReport(results
        []VerificationResult, outputDir string) error
func (v *Verifier) GenerateJSONReport(results
        []VerificationResult, outputDir string) error
func (v *Verifier) generateSummary(results []VerificationResult)
        Summary

// Config Loading
func LoadConfig(filePath string) (*config.Config, error)
func LoadConfigWithProfile(filePath, profile string)
        (*config.Config, error)

// LLM Client
func NewLLMClientWithTimeout(baseURL, apiKey string, headers
        map[string]string, timeout time.Duration) *LLMClient
```

```
func (c *LLMClient) SendMessage(ctx context.Context, model,
        message string) (string, error)
func (c *LLMClient) SendMessageWithSystem(ctx context.Context,
        model, system, message string) (string, error)

// Strategy
func NewStrategyBuilder() *StrategyBuilder
func (b *StrategyBuilder) WithName(name string) *StrategyBuilder
func (b *StrategyBuilder) WithWeights(w WeightConfig)
        *StrategyBuilder
func (b *StrategyBuilder) AddTest(test VerificationTest)
        *StrategyBuilder
func (b *StrategyBuilder) WithThresholds(t Thresholds)
        *StrategyBuilder
func (b *StrategyBuilder) Build() (ScoringStrategy, error)

// Reporter
func (v *Verifier) GenerateMarkdownReport(results
        []VerificationResult, outputDir string) error
func (v *Verifier) GenerateJSONReport(results
        []VerificationResult, outputDir string) error
```

C.2 Config Package: `digital.vasic.llmsverifier/config`

Types

```
type Config struct {
    Profile       string
    LLMs          []LLMConfig
    Global        GlobalConfig
    Database      DatabaseConfig
    API           APIConfig
    Concurrency   int
    Timeout       time.Duration
    Logging       LoggingConfig
    Monitoring    MonitoringConfig
    Security      SecurityConfig
    Notifications NotificationsConfig
}

type LLMConfig struct {
    Name     string
    Endpoint string
    APIKey   string
    Model    string
    Headers  map[string]string
    Features map[string]bool
}

type GlobalConfig struct {
    BaseURL, APIKey, DefaultModel string
    MaxRetries int
    RequestDelay, Timeout time.Duration
```

```
    CustomParams map[string]any
}

type DatabaseConfig struct {
    Path          string
    EncryptionKey string
}

type APIConfig struct {
    Port, JWTSecret string
    RateLimit, BurstLimit int
    RateLimitWindow int
    EnableCORS bool
    TrustedProxies string
    RateLimitByAPIKey bool
    CORSOrigins, CORSMethods, CORSHeaders string
    EnableHTTPS bool
    TLSCertFile, TLSKeyFile string
    ReadTimeout, WriteTimeout int
    MaxHeaderBytes int
}

type LoggingConfig struct {
    Level, Format, Output, FilePath string
    MaxSize, MaxBackups, MaxAge int
    Compress bool
}

type MonitoringConfig struct {
    EnableMetrics, EnableHealth, EnableTracing, EnableProfiling
        bool
    MetricsPort, HealthPort, TracingEndpoint, ProfilingPort string
}

type SecurityConfig struct {
    EnableRateLimiting, EnableIPWhitelist, EnableRequestLogging
        bool
    IPWhitelist []string
    EnableCSRFProtection bool
    CSRFTokenLength int
    SessionTimeout int
    SensitiveHeaders []string
}

type NotificationsConfig struct {
    Slack    SlackConfig
    Email    EmailConfig
    Telegram TelegramConfig
    Matrix   MatrixConfig
    WhatsApp WhatsAppConfig
}
```

Functions

```
func LoadFromFile(path string) (*Config,
        error)  // Supports YAML, JSON, TOML
func SaveToFile(cfg *Config, path string) error
```

C.3 Providers Package: `digital.vasic.llmsverifier/providers`

Types

```
type BaseAdapter struct {
    client *http.Client
    endpoint string
    apiKey string
    headers map[string]string
}
// Methods: Set/Get Client, Endpoint, APIKey, Headers; AddHeader

type ProviderConfig struct {
    Name, Endpoint, AuthType, StreamingFormat, DefaultModel string
    RateLimits  RateLimitConfig
    Timeouts    TimeoutConfig
    RetryConfig RetryConfig
    Features    map[string]interface{}
}

type ProviderRegistry struct { /* providers map */ }

// 25+ Provider Adapters (all embed BaseAdapter):
// OpenAIAdapter, AnthropicAdapter, GroqAdapter, DeepSeekAdapter,
// CohereAdapter, MistralAdapter, XAIAdapter, ReplicateAdapter,
// CerebrasAdapter, CloudflareAdapter, SiliconFlowAdapter,
        HyperbolicAdapter,
// TogetherAIAdapter, SambanovaAdapter, KiloAdapter, KimiAdapter,
// QwenAdapter, NovitaAdapter, NLPCloudAdapter, NIAAdapter,
// UpstageAdapter, SarvamAdapter, PublicAIAdapter, ZhipuAdapter,
// ModalAdapter, VulavulaAdapter, KimiCodeAdapter

type ModelProviderService struct { /* 3-tier discovery */ }

type ProviderClient struct {
    ProviderID, BaseURL, APIKey string
    HTTPClient *http.Client
}

type Model struct {
    ID, Name, ProviderID, ProviderName, DisplayName string
    Features, Metadata map[string]interface{}
    MaxTokens, ContextWindow int
    CostPer1MInput, CostPer1MOutput float64
    SupportsBrotli, SupportsHTTP3, SupportsToon, IsFree,
        IsOpenSource bool
```

```
    Source string  // "config", "api", "models.dev"
}

type ProviderError struct {
    Provider, Code, Message string
    Type ErrorType
    HTTPStatus int
    Retryable bool
    RetryAfter time.Duration
    RawResponse []byte
}

type VerifiedConfigGenerator struct { /* generates verified-only
        configs */ }
type VerifiedConfig struct { /* GeneratedAt, Providers,
        VerifiedModels */ }
```

Key Functions

```
// Provider Registry
func NewProviderRegistry() *ProviderRegistry
func (pr *ProviderRegistry) GetConfig(name string)
        (*ProviderConfig, bool)
func (pr *ProviderRegistry) RegisterProvider(config
        *ProviderConfig)

// Model Provider Service (3-tier discovery)
func NewModelProviderService(configPath string, logger
        *logging.Logger) *ModelProviderService
func (mps *ModelProviderService) RegisterProvider(providerID,
        baseURL, apiKey string)
func (mps *ModelProviderService) GetAllModelsWithVerification(ctx
        context.Context) (map[string][]Model, error)
func (mps *ModelProviderService) GetModels(ctx context.Context,
        providerID string) ([]Model, error)

// Verified Config Generation
func NewVerifiedConfigGenerator(service
        *EnhancedModelProviderService, logger *logging.Logger,
        outputDir string) *VerifiedConfigGenerator
func (vcg *VerifiedConfigGenerator) GenerateVerifiedConfig()
        (*VerifiedConfig, error)

// Error Handling
func NewErrorClassifier(provider string) *ErrorClassifier
func (ec *ErrorClassifier) ClassifyError(resp *http.Response,
        body []byte) *ProviderError
```

C.4 Verification Package: `digital.vasic.llmsverifier/verification`

Types

```
type Request struct { ModelID string; Prompt string }

type Verifier struct { db *database.Database }

// Code Verification
type CodeVerificationService struct {
    httpClient *client.HTTPClient
    logger     *logging.Logger
}
type CodeVerificationRequest struct {
    ModelID, ProviderID, Code, Language string
}
type CodeVerificationResponse struct {
    ModelID, ProviderID string
    Verified, CanSeeCode, AffirmativeResponse bool
    CodeUnderstanding float64
    ResponseTime int64
}
type CodeVerificationResult struct {
    VerificationID, ModelID, ProviderID, Status string
    CodeVisibility, ToolSupport, AffirmativeConfirmation bool
    ResponseAnalysis CodeResponseAnalysis
    VerificationScore float64
}
type CodeResponseAnalysis struct {
    ContainsAffirmative, ContainsNegative bool
    CodeReferences []string
    LanguageDetection, UnderstandingLevel string
    ConfidenceScore float64
}

// Coding Capability Verification
type CodingCapabilityVerificationService struct { httpClient,
        logger }
type CodingCapabilityRequest struct {
    ModelID, ProviderID, TestType, TestInput string
    ExpectedHints []string
}
type CodingCapabilityResponse struct {
    ModelID, ProviderID, TestType string
    Passed bool; Response string
    MatchedKeywords, ExpectedKeywords []string
    CapabilityScore float64; ResponseTime int64
}
type CodingCapabilityResult struct {
    VerificationID, ModelID, ProviderID, Status string
```

```
    CodebaseDetection, LanguageDetection, CodeGeneration,
        CodeAnalysis CodingCapabilityResponse
    OverallCapabilityScore float64
    CanDetectCodebase, CanIdentifyLanguage, CanGenerateCode,
        CanAnalyzeCode bool
    ReadyForCoding bool; ReadinessScore float64
}

// Provider Client Interface
type ProviderClientInterface interface {
    SendPrompt(ctx context.Context, modelID, prompt string)
        (string, error)
    StreamPrompt(ctx context.Context, modelID, prompt string)
        (<-chan string, <-chan error)
    GetModelInfo(ctx context.Context, modelID string)
        (map[string]interface{}, error)
}
```

Functions

```
func NewVerifier(db *database.Database) *Verifier
func (v *Verifier) Verify(ctx context.Context, req *Request)
        (*database.VerificationResult, error)

func NewCodeVerificationService(httpClient *client.HTTPClient,
        logger *logging.Logger) *CodeVerificationService
func (cvs *CodeVerificationService) VerifyModelCodeVisibility(ctx
        context.Context, modelID, providerID string, client
        ProviderClientInterface) (*CodeVerificationResult, error)

func NewCodingCapabilityVerificationService(httpClient
        *client.HTTPClient, logger *logging.Logger)
        *CodingCapabilityVerificationService
func (cvs *CodingCapabilityVerificationService)
        VerifyModelCodingCapabilities(ctx context.Context,
        modelID, providerID string, client
        ProviderClientInterface) (*CodingCapabilityResult, error)
```

C.5 Scoring Package: `digital.vasic.llmsverifier/scoring`

Types

```
type ScoringEngine struct {
    modelsDevClient ModelsDevClientInterface
    dbIntegration   *DatabaseIntegration
    weights         ScoreWeights
}
// Engine = alias for ScoringEngine

type ComprehensiveScore struct {
    ModelID, ModelName string
    OverallScore float64
    ScoreSuffix string
```

```
    Components ScoreComponents
    LastCalculated time.Time
    CalculationHash string
    DataSource string
}

type ScoreComponents struct {
    SpeedScore, EfficiencyScore, CostScore, CapabilityScore,
        RecencyScore float64
}

type ScoreWeights struct {
    ResponseSpeed, ModelEfficiency, CostEffectiveness,
        Capability, Recency float64
}

type ScoreThresholds struct { MinScore, MaxScore float64 }

type ScoringConfig struct {
    ConfigName string
    Weights    ScoreWeights
    Thresholds ScoreThresholds
    Enabled    bool
}

type ModelData struct {
    ID, Name, Provider, Description string
    Capabilities []string
    ContextWindow, MaxTokens int
    InputTokenCost, OutputTokenCost float64
    ThroughputRPS float64; LatencyMs int
    ReleaseDate, TrainingCutoff, LastUpdated time.Time
    ParameterCount int64
    OpenSource, Multimodal, Reasoning bool
}

type ModelRanking struct {
    Rank int; ModelID, ModelName string
    OverallScore float64; ScoreSuffix string
    Category string; CategoryScore float64
}

type ModelsDevClientInterface interface {
    FetchAllModels(ctx context.Context) (*ModelsDevAPIResponse,
        error)
    FetchModelByID(ctx context.Context, modelID string)
        (*ModelsDevModel, error)
    FetchModelsByProvider(ctx context.Context, providerID string)
        ([]ModelsDevModel, error)
}
```

Functions

```
func NewScoringEngine(db *database.Database, modelsDevClient
        ModelsDevClientInterface, logger interface{})
        *ScoringEngine
func (se *ScoringEngine) CalculateComprehensiveScore(ctx
        context.Context, modelID string, config ScoringConfig)
        (*ComprehensiveScore, error)
func (se *ScoringEngine) calculateResponseSpeedScore(modelInfo
        *ModelData, dbModel *database.Model) float64
func (se *ScoringEngine) calculateModelEfficiencyScore(modelInfo
        *ModelData, dbModel *database.Model) float64
func (se *ScoringEngine)
        calculateCostEffectivenessScore(modelInfo *ModelData,
        dbModel *database.Model) float64
func (se *ScoringEngine) calculateCapabilityScore(modelInfo
        *ModelData, dbModel *database.Model) float64
func (se *ScoringEngine) calculateRecencyScore(modelInfo
        *ModelData, dbModel *database.Model) float64

func DefaultScoringConfig() ScoringConfig  // Returns default
        weights (ResponseSpeed: 0.25, ModelEfficiency: 0.20,
        CostEffectiveness: 0.25, Capability: 0.20, Recency: 0.10)
func DefaultScoreWeights() ScoreWeights
```

C.6 API Package: `digital.vasic.llmsverifier/api`

Types

```
type Server struct {
    config   *config.Config
    database *database.Database
    server   *http.Server
}
```

API Endpoints

```
GET  /api/health              - Health check
GET  /api/models              - List models (with filtering)
GET  /api/models/{id}          - Get specific model
POST /api/models/{id}/verify   - Verify a model
GET  /api/providers            - List providers
POST /api/providers            - Add provider
```

Functions

```
func NewServer(cfg *config.Config, db *database.Database) *Server
func (s *Server) Start(port string) error
func (s *Server) Stop() error
func (s *Server) Router() http.Handler
```

C.7 Database Package: `digital.vasic.llmsverifier/database`

Types

```
type Database struct { conn *sql.DB }

// Model types
type Provider struct {
    ID, Name, Endpoint, APIKeyEncrypted, Description, Website
        string
    SupportEmail, DocumentationURL string
    CreatedAt, UpdatedAt time.Time; LastChecked *time.Time
    IsActive bool; ReliabilityScore float64;
        AverageResponseTimeMs int
}

type Model struct {
    ID int64; ModelID, Name, Description, Version string
    ProviderID int64; VerificationStatus string
    OverallScore float64; Deprecated bool
    CreatedAt, UpdatedAt time.Time
    // Capabilities (many boolean fields)
    SupportsStreaming, SupportsJSONMode, SupportsFunctionCalling
        bool
    // Pricing
    InputPricePer1M, OutputPricePer1M float64
    // Context
    MaxTokens, ContextWindow int
    // Code capabilities
    SupportsCodeGeneration, SupportsCodeCompletion,
        SupportsCodeReview bool
}

type VerificationResult struct {
    ID int64; ModelID int64; VerificationType, Status string
    StartedAt time.Time; CompletedAt *time.Time
    ModelExists, Responsive, Overloaded *bool
    LatencyMs *int
    // Feature flags (40+ boolean fields)
    SupportsToolUse, SupportsFunctionCalling,
        SupportsCodeGeneration bool
    SupportsStreaming, SupportsJSONMode, SupportsStructuredOutput
        bool
    // Scores
    CodeQualityScore, LogicCorrectnessScore,
        RuntimeEfficiencyScore float64
    OverallScore, CodeCapabilityScore, ResponsivenessScore float64
    ReliabilityScore, FeatureRichnessScore float64
}
```

CRUD Functions

```
// Provider CRUD
CreateProvider, GetProvider, GetProviderByName, UpdateProvider,
        DeleteProvider, ListProviders

// Model CRUD
CreateModel, GetModel, UpdateModel, DeleteModel, ListModels

// Verification CRUD
CreateVerificationResult, GetVerificationResult,
        ListVerificationResults, GetLatestVerificationResults,
        UpdateVerificationResult, DeleteVerificationResult

// API Keys
CreateAPIKey, GetAPIKey, UpdateAPIKey, DeleteAPIKey, ListAPIKeys

// Events
CreateEvent, GetEvent, ListEvents, UpdateEvent, DeleteEvent

// Issues
CreateIssue, GetIssue, ListIssues, UpdateIssue, DeleteIssue

// Schedules
CreateSchedule, GetSchedule, ListSchedules, UpdateSchedule,
        DeleteSchedule

// Pricing
CreatePricing, GetPricing, UpdatePricing, DeletePricing,
        ListPricing

// Config Exports
CreateConfigExport, GetConfigExport, ListConfigExports,
        UpdateConfigExport, DeleteConfigExport

// Users
CreateUser, GetUser, UpdateUser, DeleteUser, ListUsers

// Counts
GetModelCount, GetProviderCount, GetVerificationResultCount
```

C.8 Auth Package: `digital.vasic.llmsverifier/auth`

Types

```go
type AuthManager struct {
    jwtSecret []byte
    clients map[string]*Client
    ldapEnabled, rbacEnabled, ssoEnabled bool
    ldapManager *LDAPManager
    roles map[string][]string
    usageTracking *UsageTracker
}
```

```
type Client struct {
    ID int64; Name, Description, APIKey, APIKeyHash string
    Permissions []string; RateLimit int; IsActive bool
    CreatedAt, UpdatedAt time.Time; LastUsedAt *time.Time
}

type JWTClaims struct {
    ClientID int64; ClientName string; Permissions []string
    jwt.RegisteredClaims
}
```

Functions

```
func NewAuthManager(jwtSecret string, ldapConfig *LDAPConfig)
        (*AuthManager, error)
func (am *AuthManager) RegisterClient(name, description string,
        permissions []string, rateLimit int) (*Client, error)
func (am *AuthManager) AuthenticateClient(apiKey string)
        (*Client, error)
func (am *AuthManager) GenerateJWT(client *Client) (string, error)
func (am *AuthManager) ValidateJWT(tokenString string)
        (*JWTClaims, error)
func (am *AuthManager) CheckPermission(client *Client, permission
        string) bool
func (am *AuthManager) RecordUsage(clientID int64)
```

C.9 Client Package: `digital.vasic.llmsverifier/client`

Types

```
type Client struct { baseURL string; httpClient *http.Client;
        token string }
type HTTPClient struct {
    client *http.Client
    brotliCache map[string]BrotliCacheEntry
    metricsTracker *monitoring.MetricsTracker
}
type RateLimiter struct { /* token bucket */ }
```

Functions

```
func New(baseURL string) *Client
func (c *Client) SetToken(token string)
func (c *Client) Login(username, password string) error
func (c *Client) GetModels() ([]map[string]any, error)
func (c *Client) GetModel(id string) (map[string]any, error)

func NewHTTPClient(timeout time.Duration) *HTTPClient
func (c *HTTPClient) TestModelExists(ctx, provider, apiKey,
        modelID string) (bool, error)
```

```
func (c *HTTPClient) TestResponsiveness(ctx, provider, apiKey,
        modelID, prompt string) (time.Duration, time.Duration,
        error, string, bool, int, error)
func (c *HTTPClient) TestBrotliSupport(ctx, provider, apiKey,
        modelID string) (bool, error)
```

C.10 Capabilities Package: `digital.vasic.llmsverifier/capabilities`

Types (Enums)

```
// Streaming types
StreamingTypeSSE, StreamingTypeWebSocket, StreamingTypeAsyncGen,
StreamingTypeJSONL, StreamingTypeMpscStream,
        StreamingTypeEventStream,
StreamingTypeStdout, StreamingTypeNone

// HTTP versions
HTTPVersion1_1, HTTPVersion2, HTTPVersion3

// Compression types
CompressionGzip, CompressionBrotli, CompressionDeflate,
        CompressionZstd,
CompressionSemantic, CompressionChat, CompressionNone

// Caching types
CachingAnthropic, CachingDashScope, CachingPrompt,
        CachingSemantic, CachingLLMOps, CachingNone

// Protocol types
ProtocolMCP, ProtocolACP, ProtocolLSP, ProtocolGRPC,
ProtocolOpenAI, ProtocolAnthropic, ProtocolOllama

// Auth types
AuthAPIKey, AuthBearer, AuthOAuth2, AuthNone, AuthAWSSigV4

// Main capability struct
type ProviderCapabilities struct {
    Provider, Model string; Verified bool; VerifiedAt time.Time
    Streaming StreamingCapability
    Network NetworkCapability
    Compression CompressionCapability
    Caching CachingCapability
    Protocols []ProtocolType
    Auth AuthCapability
    Model_ ModelCapability
    Extended ExtendedCapabilities
    Custom map[string]interface{}
}
```

D. VALIDATION SYSTEM

How Models Are Validated

1. **Existence Check**: Verify the model exists on the provider's API (`TestModelExists`)
2. **Responsiveness Check**: Send a test prompt and measure response time (`TestResponsiveness`)
3. **Feature Detection**: Dynamically detect supported features (streaming, function calling, etc.)
4. **Code Visibility Verification** (MANDATORY): Ask "Do you see my code?" - model must respond affirmatively
5. **Coding Capability Verification**: Test practical coding abilities:
   - Codebase detection
   - Language identification
   - Code generation
   - Code analysis
6. **Capability Scoring**: Score each capability dimension

Validation Types (from api/validation.go)

Custom validators registered:

- `alphanumspace` - Alphanumeric with spaces
- `url` - URL validation
- `email` - Email validation
- `cron` - Cron expression validation
- `severity` - One of: info, warning, error, critical
- `event_type` - Predefined event types
- `verification_type` - Verification type validation
- `status` - Status validation
- `schedule_type` - Schedule type validation
- `target_type` - Target type validation
- `export_type` - Export type validation
- `issue_type` - Issue type validation
- `pricing_model` - Pricing model validation
- `limit_type` - Limit type validation
- `reset_period` - Reset period validation
- `port` - Port number validation

E. VERIFICATION SYSTEM

Core Verification Flow

```
1. Load Configuration (config.yaml)
   |
2. For each configured LLM:
   |
   +-- 3. Test Model Existence
```

```
|      - Query provider's /models endpoint
|      - Verify model ID is in response
|
+-- 4. Test Responsiveness
|      - Send simple prompt
|      - Measure response time
|      - Check for valid response
|
+-- 5. Feature Detection
|      - Test streaming support (SSE, WebSocket)
|      - Test function calling
|      - Test JSON mode
|      - Test vision capabilities
|      - Test embeddings
|
+-- 6. Code Visibility Verification (MANDATORY)
|      - Send code snippet
|      - Ask "Do you see my code?"
|      - Check affirmative response
|      - Verify code references in response
|
+-- 7. Coding Capability Verification
|      - Test codebase detection
|      - Test language identification
|      - Test code generation
|      - Test code analysis
|
+-- 8. Calculate Scores
       - Response speed score
       - Model efficiency score
       - Cost effectiveness score
       - Capability score
       - Recency score
       - Weighted overall score
```

Verification Types

- **Standard Verification**: Basic existence + responsiveness + features
- **Code Verification**: Mandatory code visibility test
- **Coding Capability**: Practical coding task evaluation
- **Comprehensive Verification**: All tests combined
- **Relaxed Verification**: Less strict requirements (configurable)

F. SCORING SYSTEM

Score Components (from scoring/types.go)

```go
type ScoreWeights struct {
    ResponseSpeed      float64  // 0.25 default - How fast the
        model responds
```

```
    ModelEfficiency    float64  // 0.20 default - Token
        throughput, resource usage
    CostEffectiveness  float64  // 0.25 default - Price per token
    Capability         float64  // 0.20 default - Feature support
        breadth
    Recency            float64  // 0.10 default - How recently
        updated
}
```

Scoring Algorithm

```
Overall Score = (SpeedScore * 0.25) +
                (EfficiencyScore * 0.20) +
                (CostScore * 0.25) +
                (CapabilityScore * 0.20) +
                (RecencyScore * 0.10)
```

Scores are clamped to range [0, 10].

Individual Score Calculations

1. **Response Speed Score** (`calculateResponseSpeedScore`):
    - Based on latency measurements
    - Faster = higher score
    - Uses models.dev API data + live tests
2. **Model Efficiency Score** (`calculateModelEfficiencyScore`):
    - Based on context window size, throughput
    - Parameter count consideration
3. **Cost Effectiveness Score** (`calculateCostEffectivenessScore`):
    - Input/output token pricing
    - Free/open source bonus
4. **Capability Score** (`calculateCapabilityScore`):
    - Number of supported features
    - Streaming, function calling, vision, etc.
    - Protocol support (MCP, ACP, LSP)
5. **Recency Score** (`calculateRecencyScore`):
    - Release date / last update
    - Newer models score higher

Score Output Format

```
{
  "model_id": "gpt-4",
  "model_name": "GPT-4",
```

```
  "overall_score": 8.5,
  "score_suffix": "(SC:8.5)",
  "components": {
    "speed_score": 7.2,
    "efficiency_score": 8.8,
    "cost_score": 6.5,
    "capability_score": 9.2,
    "recency_score": 9.5
  }
}
```

G. PROVIDER SUPPORT

Supported Providers (25+)

Provider	File	Auth Type	Endpoint	Notes
OpenAI	openai.go	Bearer	api.openai.com/v1	Full API support
Anthropic	anthropic.go	API Key + OAuth	api.anthropic.com	OAuth via Claude CLI
Groq	groq.go	Bearer	api.groq.com/openai/v1	Streaming SSE
DeepSeek	deepseek.go	Bearer	api.deepseek.com	
Cohere	cohere.go	Bearer	api.cohere.com	
Mistral	mistral.go	Bearer	api.mistral.ai	
xAI	xai.go	Bearer	api.x.ai	Grok models
Replicate	replicate.go	Token	api.replicate.com	
Cerebras	cerebras.go	Bearer	api.cerebras.ai	
Cloudflare Workers AI	cloudflare.go	API Key	api.cloudflare.com	
SiliconFlow	siliconflow.go	Bearer	api.siliconflow.cn	
Hyperbolic	hyperbolic.go	Bearer	api.hyperbolic.xyz	
Together AI	togetherai.go	Bearer	api.together.xyz	
SambaNova	sambanova.go	Bearer	api.sambanova.ai	
KiloCode	kilo.go	Bearer	Various	
Moonshot/ Kimi	kimi.go	Bearer	api.moonshot.cn	
Qwen/ DashScope	qwen.go	API Key	dashscope.aliyuncs.com	
Novita AI	novita.go	Bearer	api.novita.ai	
NLP Cloud	nlpcloud.go	Token	api.nlpcloud.io	

Provider	File	Auth Type	Endpoint	Notes
Upstage	upstage.go	Bearer	api.upstage.ai	
Sarvam	sarvam.go	API Key	api.sarvam.ai	
Zhipu	zhipu.go	API Key	open.bigmodel.cn	
Modal	modal.go	Token	modal.com	
Public AI	publicai.go	API Key	api.publicai.net	
NIA	nia.go	API Key	api.nia.ai	
Vulavula	vulavula.go	API Key	api.vulavula.com	
KimiCode	kimicode.go	Bearer	api.kimicode.com	

Provider Configuration (per provider)

- **Auth Type**: bearer, api_key, oauth
- **Rate Limits**: requests/minute, requests/hour, burst limit
- **Timeouts**: request, stream, connect
- **Retry Config**: max retries, initial delay, max delay, backoff factor, retryable errors
- **Features**: streaming, functions, vision, ACP, max context, supported models

3-Tier Model Discovery

1. **Priority 1**: User configuration files
2. **Priority 2**: Provider API (live model listing)
3. **Priority 3**: models.dev fallback (cached)

H. CONFIGURATION SYSTEM

Configuration File Format (YAML)

```yaml
profile: "production"  # dev, prod, test

global:
  base_url: "https://api.openai.com/v1"
  api_key: "${OPENAI_API_KEY}"  # Environment variable expansion
  max_retries: 3
  request_delay: 1s
  timeout: 30s

database:
  path: "./llm-verifier.db"
```

```yaml
  encryption_key: "your-encryption-key"  # SQL Cipher

api:
  port: "8080"
  jwt_secret: "your-jwt-secret"
  rate_limit: 100
  burst_limit: 20
  enable_cors: true
  cors_origins: "http://localhost:3000"
  enable_https: false
  tls_cert_file: ""
  tls_key_file: ""

# LLM configurations (optional - auto-discovery if empty)
llms:
  - name: "OpenAI GPT-4"
    endpoint: "https://api.openai.com/v1"
    api_key: "${OPENAI_API_KEY}"
    model: "gpt-4-turbo"
    headers:
      Custom-Header: "value"
    features:
      tool_calling: true
      embeddings: false

concurrency: 5
timeout: 60s

logging:
  level: "info"      # debug, info, warn, error
  format: "json"     # json, text
  output: "stdout"   # stdout, stderr, file
  file_path: "/var/log/llm-verifier.log"
  max_size: 100
  max_backups: 5
  max_age: 30
  compress: true

monitoring:
  enable_metrics: true
  metrics_port: "9090"
  enable_health: true
  health_port: "8081"
  enable_tracing: false
  enable_profiling: false

security:
  enable_rate_limiting: true
  enable_ip_whitelist: false
  ip_whitelist: []
  enable_request_logging: true
  enable_csrf_protection: false
  session_timeout: 60
```

```yaml
notifications:
  slack:
    enabled: false
    webhook_url: ""
  email:
    enabled: false
    smtp_host: ""
    smtp_port: 587
    username: ""
    password: ""
  telegram:
    enabled: false
    bot_token: ""
    chat_id: ""
```

Required API Keys/Credentials

- **OPENAI_API_KEY**: For OpenAI provider
- **ANTHROPIC_API_KEY**: For Anthropic provider
- Provider-specific API keys for each configured provider
- **JWT_SECRET**: For API authentication
- **DATABASE_ENCRYPTION_KEY**: For SQL Cipher (optional)

Environment Variables

- `LLM_VERIFIER_*`: All config values can be overridden via env vars
- Provider API keys: `${PROVIDER_API_KEY}` syntax in config
- `LLM_DB_PATH`: Database file path
- `PORT` / `GIN_MODE`: Server settings

I. TEST SUITE

Test Structure (across all packages)

- **Total Go files**: 512
- **Core Go files in llm-verifier/**: 448
- **Test files**: ~130+ (roughly 25% of source files)

Test Types

```
llm-verifier/
├── llmverifier/
│   ├── verifier_test.go
│   ├── verifier_core_final_test.go
│   ├── verifier_ultimate_test.go
│   ├── verifier_comprehensive_test_fixed.go
│   ├── strategy_test.go
│   ├── strategy_integration_test.go
```

```
│   ├── strategy_security_test.go
│   ├── strategy_stress_test.go
│   ├── strategy_e2e_test.go
│   ├── strategy_builder_test.go
│   ├── feature_detection_test.go
│   ├── config_export_test.go
│   ├── config_loader_test.go
│   ├── config_migration_test.go
│   ├── llm_client_test.go
│   ├── models_test.go
│   ├── reporter_test.go
│   ├── analytics_test.go
│   └── integration_test.go
├── providers/
│   ├── providers_test.go
│   ├── providers_extended_test.go
│   ├── openai_test.go
│   ├── anthropic_test.go
│   ├── deepseek_extended_test.go
│   ├── model_provider_service_test.go
│   ├── model_verification_test.go
│   ├── verified_config_generator_test.go
│   ├── adapters_test.go
│   ├── fallback_models_test.go
│   └── integration_test.go
├── verification/
│   ├── verification_test.go
│   ├── verification_real_test.go
│   ├── code_verification_test.go
│   ├── code_verification_integration_test.go
│   ├── coding_capability_verification_test.go
│   └── meaningful_response_test.go
├── scoring/
│   ├── scoring_engine_test.go
│   ├── types_test.go
│   ├── metrics_collector_test.go
│   ├── alert_manager_test.go
│   ├── model_display_test.go
│   ├── model_naming_test.go
│   ├── database_extensions_test.go
│   └── integration_test.go
├── api/
│   ├── handlers_test.go
│   ├── handlers_integration_test.go
│   ├── middleware_test.go
│   ├── server_test.go
│   ├── server_init_test.go
│   ├── validation_test.go
│   ├── types_test.go
│   ├── errors_sanitize_test.go
│   ├── schema_validator_test.go
│   └── security_middleware_test.go
├── database/
│   ├── crud_test.go
```

```
│   ├── database_helpers_test.go
│   ├── migrations_test.go
│   ├── optimizations_test.go
│   ├── in_memory_test.go
│   ├── events_crud_test.go
│   ├── issues_crud_test.go
│   ├── logs_crud_test.go
│   ├── config_exports_crud_test.go
│   └── verification_scores_crud_test.go
├── config/
│   ├── config_test.go
│   ├── loader_test.go
│   └── validation_test.go
├── auth/
│   ├── auth_manager_test.go
│   ├── compliance_test.go
│   ├── ldap_test.go
│   └── rbac_test.go
├── client/
│   ├── client_test.go
│   ├── client_manager_test.go
│   ├── http_client_test.go
│   └── rate_limiter_test.go
├── ai/
│   └── simple_recommender_test.go
├── analytics/
│   └── predictive_test.go
└── capabilities/
    └── capabilities_test.go
```

External Tests

```
tests/
├── unit/              # Unit tests
├── integration/       # Integration tests
├── e2e/              # End-to-end tests
├── performance/       # Performance benchmarks
├── security/          # Security tests
├── automation/        # Automated test suites
└── challenges/        # Challenge-based tests
```

J. DOCUMENTATION

Main Documentation Files

File	Purpose
README.md	Main project documentation
API_REFERENCE.md	Complete API reference

File	Purpose
ACP_API_DOCUMENTATION.md	AI Coding Protocol API docs
ACP_EXAMPLES_AND_DEMOS.md	Usage examples and demos
ACP_IMPLEMENTATION_DESIGN.md	Implementation design
AGENTS.md	CLI agent documentation
SECURITY.md	Security configuration
VERIFYING.md	Verification guide
DEPLOYMENT.md	Deployment guide
GETTING_STARTED.md	Quick start guide
DEVELOPER_GUIDE.md	Developer documentation
USER_MANUAL.md	User manual
CONSTITUTION.md	Project constitution
CODE_OF_CONDUCT.md	Code of conduct
CONTRIBUTING.md	Contribution guidelines

In-Project Documentation

```
llm-verifier/docs/
├── COMPLETE_USER_MANUAL.md
├── USER_MANUAL.md
├── API_DOCUMENTATION.md
├── DEPLOYMENT_GUIDE.md
├── ENVIRONMENT_VARIABLES.md
├── CHANGELOG.md
├── bigdata/
└── deployment/

docs/
├── ARCHITECTURE_OVERVIEW.md
├── COMPLETE_SYSTEM_DOCUMENTATION.md
├── MODEL_VERIFICATION_GUIDE.md
├── LLMSVD_SUFFIX_GUIDE.md
├── CONFIGURATION_MIGRATION_GUIDE.md
├── CAPABILITY_DETECTION.md
├── COMPREHENSIVE_TEST_SUITE_DOCUMENTATION.md
├── scoring/
│   ├── guides/
│   ├── tutorials/
│   ├── examples/
│   └── api/
└── monitoring/
```

K. DOCKER SUPPORT

Dockerfile (Multi-Stage)

```
Stage 1: Build (golang:1.21-alpine)
  - Install git, ca-certificates, tzdata
  - Download dependencies
  - Build static binary with CGO_ENABLED=0

Stage 2: Security Scan (aquasecurity/trivy)
  - Run Trivy security scanner
  - Output scan results as JSON

Stage 3: Runtime (gcr.io/distroless/static-debian12)
  - Copy CA certificates
  - Copy timezone data
  - Copy binary and scan results
  - Run as non-root (UID 65534)
  - Health check: /llm-verifier health
  - Expose port 8080
```

docker-compose.yml

- Main service with memory limits (4GB), PID limits (2048)
- Health check every 30s via wget
- Volume mounts for data, exports, config
- Optional PostgreSQL and Redis services (commented)
- Custom bridge network

docker-compose.prod.yml

- Production-optimized configuration

docker-compose.messaging.yml

- Kafka and RabbitMQ messaging stack

L. DEPENDENCIES

Core Go Dependencies (from go.mod)

Package	Version	Purpose
github.com/gin-gonic/gin	v1.11.0	HTTP web framework
github.com/spf13/cobra	v1.10.2	CLI framework
github.com/spf13/viper	v1.21.0	Configuration management

Package	Version	Purpose
github.com/charmbracelet/bubbletea	v1.1.0	TUI framework
github.com/charmbracelet/lipgloss	v0.13.0	Terminal styling
github.com/mattn/go-sqlite3	v1.14.32	SQLite driver (CGO)
github.com/golang-jwt/jwt/v5	v5.3.0	JWT authentication
github.com/go-ldap/ldap/v3	v3.4.12	LDAP integration
github.com/go-playground/validator/v10	v10.27.0	Request validation
github.com/gorilla/websocket	v1.5.3	WebSocket support
github.com/google/uuid	v1.6.0	UUID generation
github.com/andybalholm/brotli	v1.2.0	Brotli compression
golang.org/x/crypto	v0.46.0	Argon2, crypto utilities
cloud.google.com/go/storage	v1.58.0	Google Cloud Storage
github.com/aws/aws-sdk-go-v2	v1.41.0	AWS SDK
github.com/Azure/azure-sdk-for-go/sdk/storage/azblob	v1.6.3	Azure Blob Storage
github.com/minio/minio-go/v7	v7.0.98	MinIO/S3 compatible
google.golang.org/api	v0.256.0	Google APIs
google.golang.org/grpc	v1.76.0	gRPC support
github.com/stretchr/testify	v1.11.1	Testing framework
github.com/swaggo/swag	v1.16.6	Swagger/OpenAPI docs
github.com/rabbitmq/amqp091-go	v1.10.0	RabbitMQ client
github.com/segmentio/kafka-go	v0.4.50	Kafka client
github.com/sirupsen/logrus	v1.9.4	Structured logging

Python SDK Dependencies

```
llm-verifier-sdk/
├── Client class
├── Async support
├── Type hints
├── Error handling
```

M. INTEGRATION PATTERN

How to Integrate LLMsVerifier into HelixTranslate

Option 1: Go Module Import (Recommended)

```
// go.mod
require digital.vasic.llmsverifier v0.0.0
```

```
replace digital.vasic.llmsverifier => ./
        llm-verifier  // Or remote path

// In HelixTranslate code:
package main

import (
    "digital.vasic.llmsverifier/config"
    "digital.vasic.llmsverifier/llmverifier"
    "digital.vasic.llmsverifier/providers"
    "digital.vasic.llmsverifier/verification"
    "digital.vasic.llmsverifier/scoring"
    "digital.vasic.llmsverifier/database"
)

func main() {
    // 1. Load configuration
    cfg, err := config.LoadFromFile("config.yaml")
    if err != nil {
        log.Fatal(err)
    }

    // 2. Initialize database
    db, err := database.New(cfg.Database.Path,
        cfg.Database.EncryptionKey)
    if err != nil {
        log.Fatal(err)
    }

    // 3. Create provider registry
    registry := providers.NewProviderRegistry()

    // 4. Register providers with API keys
    for _, llm := range cfg.LLMs {
        // Register each provider
    }

    // 5. Create verifier
    verifier := llmverifier.New(cfg)

    // 6. Run verification
    results, err := verifier.Verify()
    if err != nil {
        log.Fatal(err)
    }

    // 7. Score models
    engine := scoring.NewScoringEngine(db, modelsDevClient,
        logger)
    for _, result := range results {
        score, err := engine.CalculateComprehensiveScore(ctx,
        result.ModelInfo.ID, scoring.DefaultScoringConfig())
        // Use score data...
```

```
    }

    // 8. Generate reports
    verifier.GenerateMarkdownReport(results, "./reports")
    verifier.GenerateJSONReport(results, "./reports")
}
```

Option 2: REST API Client

```
import "digital.vasic.llmsverifier/client"

// Create client
cli := client.New("http://localhost:8080")

// Authenticate
cli.Login("username", "password")

// List models
models, err := cli.GetModels()

// Verify a model
// (POST /api/models/{id}/verify)
```

Option 3: Direct Package Usage (Selective)

```
// Use only specific packages

// For provider management
import "digital.vasic.llmsverifier/providers"
mps := providers.NewModelProviderService("config.yaml", logger)
models, err := mps.GetAllModelsWithVerification(ctx)

// For verification only
import "digital.vasic.llmsverifier/verification"
verifier := verification.NewVerifier(db)
result, err := verifier.Verify(ctx,
        &verification.Request{ModelID: "gpt-4", Prompt: "test"})

// For scoring only
import "digital.vasic.llmsverifier/scoring"
engine := scoring.NewScoringEngine(db, client, logger)
score, err := engine.CalculateComprehensiveScore(ctx, "gpt-4",
        scoring.DefaultScoringConfig())

// For capability detection
import "digital.vasic.llmsverifier/capabilities"
detector := capabilities.NewDetector()
caps, err := detector.DetectProviderCapabilities(ctx, "openai",
        apiKey)
```

Configuration for HelixTranslate

```yaml
# helix-verifier-config.yaml
profile: "production"

global:
  max_retries: 3
  timeout: 30s

database:
  path: "./helix-verifier.db"

llms:
  - name: "Helix-Primary"
    endpoint: "${HELIX_API_ENDPOINT}"
    api_key: "${HELIX_API_KEY}"
    model: "${HELIX_MODEL}"
    features:
      code_generation: true
      code_completion: true
      streaming: true

verification:
  enabled: true
  strict_mode: true
  require_affirmative: true
  max_retries: 3
  timeout_seconds: 30
  min_verification_score: 0.7

scoring:
  weights:
    response_speed: 0.30
    model_efficiency: 0.25
    cost_effectiveness: 0.20
    capability: 0.20
    recency: 0.05
```

Key Integration Points

1. **Import the module**: `digital.vasic.llmsverifier` as a Go module dependency
2. **Configuration**: Load from YAML/JSON with provider-specific settings
3. **Database**: Initialize SQLite with encryption for local state
4. **Provider registration**: Register Helix's LLM providers
5. **Verification pipeline**: Run the full verification flow
6. **Scoring**: Calculate scores using the scoring engine
7. **Report generation**: Generate Markdown/JSON reports
8. **API integration**: Optionally run the REST API server alongside HelixTranslate

Critical Notes for HelixTranslate Integration

- The project uses **Go modules** - must be imported as a Go dependency
- Requires **CGO** for SQLite (mattn/go-sqlite3)
- All provider API keys should be passed via **environment variables**
- The **verification system is mandatory** - models must pass "Do you see my code?" test
- The **scoring system is pluggable** - customize weights via config
- **Database encryption** is optional but recommended for production
- **Branding suffix** `(llmsvd)` is added to all generated configs